

MIT 212 — Programming Techniques

MIT 212 — ප්‍රශ්න බැංකුව සහ පිළිතුරු — 3 සහ 4 සති

Question Bank with Model Answers • Week 3: Data Structures & Algorithmic Thinking • Week 4:
Developer Workflows (Git & GitHub)

Structure: Part A — 20 MCQs • Part B — 10 True/False with corrections • Part C — 10 definitions & short answers
• Part D — 5 comparison questions • Part E — 8 code-output questions • Part F — 6 error-identification &
debugging • Part G — 4 algorithm/pseudocode questions • Part H — 2 flowchart questions • Part I — 5 short
programming questions • Part J — 1 substantial practical problem with full model solution.

සෑම කොටසකම ප්‍රශ්නවලට පසුව ආදර්ශ පිළිතුරු ලබා දී ඇත. පළමුව පිළිතුරු නොබලා උත්සාහ
කරන්න.

Part A — Multiple-Choice Questions (20) | බහුවරණ ප්‍රශ්න 20

Choose the single best answer. — හොඳම පිළිතුර එකක් තෝරන්න.

- Q1.** A data structure is best defined as:
- A) A programming language feature for printing output
 - B) A specialized format for organizing, processing, retrieving, and storing data
 - C) A type of computer memory
 - D) A Git command
- Q2.** Which of the following contains ONLY primitive data types?
- A) list, int, bool
 - B) int, float, string, bool
 - C) tuple, set, dictionary
 - D) string, list, float
- Q3.** For numbers = [10, 20, 30], what does numbers[0] return?
- A) 0
 - B) 10
 - C) 20
 - D) IndexError
- Q4.** Which is NOT a characteristic of Python lists?
- A) Ordered
 - B) Mutable
 - C) Immutable
 - D) Allows duplicates
- Q5.** For a list of length 5, the valid indices are:
- A) 1 to 5
 - B) 0 to 5
 - C) 0 to 4
 - D) -1 to 5
- Q6.** colors[-1] returns:
- A) The first item
 - B) The last item
 - C) An IndexError always
 - D) An empty list
- Q7.** Which loop is described in the lecture as “the most elegant way to traverse” a list in Python?
- A) while loop
 - B) for i in range(len(list))
 - C) for item in list
 - D) do-while loop
- Q8.** When do we need range(len(numbers)) instead of for num in numbers?
- A) When the list is empty
 - B) When we need the position (index) while looping
 - C) When the list contains strings
 - D) Never
- Q9.** An algorithm is:
- A) Any Python program
 - B) A finite sequence of well-defined instructions to solve a specific class of problems
 - C) A flowchart symbol
 - D) A data structure
- Q10.** The correct order of the problem-solving framework is:
- A) Code → Plan → Logic → Understand
 - B) Understand → Plan → Logic → Code

- C) Plan → Code → Test → Deploy
- D) Logic → Understand → Code → Plan

Q11. In the sum-of-list algorithm, total = 0 before the loop is called:

- A) A counter
- B) An accumulator initialisation
- C) A parameter
- D) A merge

Q12. Why must max_val start as numbers[0] rather than 0?

- A) It is faster
- B) 0 gives the wrong answer for lists of all negative numbers
- C) Python requires it
- D) It prevents IndexError

Q13. Looping through a list once has time complexity:

- A) $O(1)$
- B) $O(n)$ — linear
- C) $O(n^2)$
- D) $O(\log n)$

Q14. In def find_average(numbers): the name numbers is:

- A) An argument
- B) A parameter
- C) A return value
- D) A global constant

Q15. Git is best described as:

- A) A cloud website for storing code
- B) A distributed version control system that tracks changes over time
- C) A Python IDE
- D) A Microsoft database

Q16. Git was created by:

- A) Bill Gates, 1995
- B) Guido van Rossum, 2000
- C) Linus Torvalds, April 2005
- D) Mark Zuckerberg, 2008

Q17. The correct Git workflow order is:

- A) commit → add → push
- B) add → commit → push
- C) push → commit → add
- D) clone → push → add

Q18. git pull is equivalent to:

- A) fetch + merge
- B) clone + push
- C) add + commit
- D) branch + checkout

Q19. Which command creates AND switches to a new branch?

- A) git branch new-feature
- B) git merge new-feature
- C) git checkout -b new-feature
- D) git clone new-feature

Q20. A merge conflict occurs when:

- A) Two branches modify the same line
- B) A file is too large
- C) You forget git init

D) The internet disconnects

Part A — Answer Key | පිළිතුරු

Q	Ans	Why
1	B	The lecture's exact definition; the analogy is a filing cabinet.
2	B	Primitive types store a single value; lists/tuples/sets/dictionaries are non-primitive.
3	B	Zero-based indexing: index 0 is the first element.
4	C	Lists are mutable — they can be changed after creation.
5	C	Length $n \rightarrow$ indices 0 to $n-1$; <code>nums[5]</code> raises <code>IndexError</code> .
6	B	Negative indices count from the end; -1 is the last item.
7	C	The <code>for...in</code> loop assigns each item to the loop variable temporarily.
8	B	Index-based traversal is used when the position is needed.
9	B	Example from the lecture: a sandwich recipe.
10	B	Understand inputs/outputs, plan pseudocode, define control flow, then code.
11	B	The accumulator collects the result across iterations.
12	B	For <code>[-5,-2,-9]</code> a start of 0 would wrongly return 0.
13	B	If the list doubles, the time doubles — linear time.
14	B	Parameter = name in the definition; argument = value at the call.
15	B	GitHub is the cloud website; Git is the local distributed VCS.
16	C	Created for Linux kernel development; first version in ~2 weeks.
17	B	Working Directory $\rightarrow$ Staging (<code>add</code>) $\rightarrow$ Local Repo (<code>commit</code>) $\rightarrow$ Remote (<code>push</code>).
18	A	It downloads remote changes and merges them into the local repo.
19	C	<code>git branch</code> only creates; <code>checkout -b</code> creates and switches.
20	A	Git cannot decide which version of the line to keep.

Part B — True/False with Corrections (10) | සත්‍ය/අසත්‍ය සහ නිවැරදි කිරීම්

State True or False. If False, rewrite the statement correctly. — අසත්‍ය නම් නිවැරදිව නැවත ලියන්න.

- Q1.** Python lists use one-based indexing, so the first element is list[1].
- Q2.** A Python list can hold different data types at the same time.
- Q3.** Tuples and strings are examples of mutable types.
- Q4.** Slicing colors[1:3] returns the elements at indices 1, 2 and 3.
- Q5.** An interpreter translates and executes a program line by line, and Python is interpreted.
- Q6.** A logical error is detected by Python and stops the program with an error message.
- Q7.** Git stores only the differences (diffs) between file versions.
- Q8.** git commit uploads your changes to GitHub.
- Q9.** GitHub works completely offline once installed.
- Q10.** A fast-forward merge simply moves the branch pointer forward when there are no diverging commits.

Part B — Answer Key | පිළිතුරු

Q	T/F	Correction / Explanation
1	FALSE	Lists use ZERO-based indexing — the first element is list[0].
2	TRUE	Correct — lists are dynamic ([1, "two", 3.0, True] is valid).
3	FALSE	Tuples and strings are IMMUTABLE; lists, dictionaries, and sets are mutable.
4	FALSE	The stop index is EXCLUDED — colors[1:3] returns indices 1 and 2 only.
5	TRUE	Correct — a compiler, by contrast, translates the whole program before running.
6	FALSE	A logical error is NEVER reported — the program runs but gives the wrong result. Runtime errors stop the program.
7	FALSE	Git stores COMPLETE SNAPSHOTS of the project; unchanged files are referenced, not duplicated.
8	FALSE	git commit saves a snapshot to the LOCAL repository; git push uploads to GitHub.
9	FALSE	GitHub is a cloud website and requires internet; GIT works offline.
10	TRUE	Correct — a 3-way merge is needed (creating a merge commit) when branches have diverged.

Part C — Definitions & Short-Answer Questions (10) | නිර්වචන සහ කෙටි පිළිතුරු

Q1. Define a data structure and give the real-world analogy used in the lecture.

දත්ත ව්‍යුහයක් නිර්වචනය කර දේශනයේ භාවිත උපමාව දෙන්න.

Model answer: A data structure is a specialized format for organizing, processing, retrieving, and storing data. Analogy: a filing cabinet — we don't throw papers in a pile; we use folders and labels for quick retrieval.

Q2. State the four characteristics of a Python list.

Python ලැයිස්තුවක ලක්ෂණ හතර සඳහන් කරන්න.

Model answer: 1) Ordered — elements keep their defined position. 2) Mutable — items can be changed, added, or removed after creation. 3) Allows duplicates — the same value can appear multiple times. 4) Dynamic — different data types can live in the same list.

Q3. What is traversal, and which loop is the most elegant way to traverse in Python?

Traversal යනු කුමක්ද? වඩාත් අලංකාර ලූපය කුමක්ද?

Model answer: Traversal means visiting every item in a list once. The for...in loop is the most elegant way: each item is assigned to the loop variable temporarily (for num in numbers: print(num)).

Q4. Define an algorithm and give the example from the lecture.

ඇල්ගොරිතමයක් නිර්වචනය කර දේශන උදාහරණය දෙන්න.

Model answer: An algorithm is a finite sequence of well-defined instructions designed to solve a specific class of problems. Example: a recipe for making a sandwich.

Q5. What is pseudocode and why is it used before coding?

ව්‍යාජ කේතය යනු කුමක්ද? කේත කිරීමට පෙර ඇයි?

Model answer: Pseudocode is “fake code” written for human reading — it follows the same logic as a program but is not strictly Python. It is used in the Plan step of the framework to sketch the steps and verify the logic before translating it into real code.

Q6. Explain $O(n)$ linear time using the lecture's description.

$O(n)$ රේඛීය කාලය පැහැදිලි කරන්න.

Model answer: Efficiency measures how resources change as the data size (n) grows. Looping through a list once is linear time: if the list doubles in size, the running time doubles.

Q7. Define Git and state who created it, when, and why.

Git නිර්වචනය කර නිර්මාතෘ, කවදා, ඇයි දැයි කියන්න.

Model answer: Git is a distributed version control system that tracks changes in source code over time. It was created by Linus Torvalds in April 2005 for Linux kernel development (which has 15,000+ contributing developers); he built the first version in about two weeks.

Q8. Explain the difference between the staging area and the local repository.

Staging area සහ local repository අතර වෙනස පැහැදිලි කරන්න.

Model answer: The staging area is the middle zone where edited files are placed with git add — a shortlist of what will go into the next snapshot. The local repository is where git commit -m permanently records that snapshot with a message on your machine.

Q9. What is a commit? Use the lecture's analogy.

Commit යනු කුමක්ද? උපමාව භාවිතා කරන්න.

Model answer: A commit is a saved snapshot of your project at a specific point in time — like a save game. Each commit points back to the previous one, forming a timeline of the project's history.

Q10. What is a pull request and which platform provides it?

Pull request යනු කුමක්ද? කුමන වෙබ්සයිට් එක?

Model answer: A pull request is a GitHub feature that lets a developer propose changes and have them reviewed (code review) before merging into the main branch. It is one of GitHub's collaboration features, alongside issues, project boards, and GitHub Actions.

Part D — Comparison Questions (5) | සංසන්දන ප්‍රශ්න

Write at least three differences for each pair, in a table. Model answers below each question.

Q1. Compare a for loop and a while loop, with one traversal example each.

for සහ while ලූප සංසන්දනය කරන්න.

Model answer:

Aspect	for loop	while loop
When used	Known number of iterations / visiting each item of a collection	Repeats until a condition becomes False; count unknown in advance
Counter	Managed automatically	Programmer must initialise and update it (i = 0; i += 1)
Risk	Rarely infinite	Infinite loop if the condition variable is never updated
Example	for num in nums: print(num)	i = 0 while i < len(nums): print(nums[i]); i += 1

Q2. Compare print() and return in the context of functions.

print() සහ return සංසන්දනය කරන්න.

Model answer:

Aspect	print()	return
Purpose	Displays a value on screen for the user	Sends a value back to the caller for storage/reuse
Location	Anywhere	Only inside a function; ends the function immediately
Reuse	Value cannot be used later	avg = find_average(marks) captures the value
Trap	A print-only function returns None	Unreachable code after return never runs

Q3. Compare primitive and non-primitive data structures with examples.

ප්‍රාථමික/අප්‍රාථමික සංසන්දනය.

Model answer:

Aspect	Primitive	Non-Primitive
Stores	A single value at a time	Collections of values or complex objects
Examples	int, float, string, bool	list, dictionary, tuple, set
Role	Basic building blocks	Built from primitives; manage whole collections

Q4. Compare Git and GitHub (at least four differences).

Git සහ GitHub සංසන්දනය (වෙනස් 4).

Model answer:

Aspect	Git	GitHub
Nature	Local version-control tool installed on your computer	Cloud platform / website (github.com)

Aspect	Git	GitHub
Internet	Works offline	Requires internet
Interface	Command-line tool	Web interface built on top of Git
Function	Tracks changes locally; commits & branches	Stores repos online; pull requests, code review, collaboration
Ownership	Free & open source	Owned by Microsoft

Q5. Compare the three merge strategies: fast-forward, 3-way, and squash.

ඒකාබද්ධ උපාය තුන සංසන්දනය කරන්න.

Model answer:

Strategy	How it works	When used
Fast-forward	No merge commit — Git simply moves the branch pointer forward	When there are no diverging commits on the target branch
3-way merge	Creates a merge commit joining two histories	When the branches have diverged
Squash merge	Combines all feature commits into one clean commit	To keep main history tidy when merging a feature branch

Part E — Code-Output Questions (8) | කේත ප්‍රතිදාන ප්‍රශ්න

Write the exact output of each code segment. — එක් එක් කේත කොටසේ නිශ්චිත ප්‍රතිදානය ලියන්න.

Q1.

```
numbers = [10, 20, 30]
print(numbers[0])
numbers[2] = 100
print(numbers)
```

Answer: 10 | [10, 20, 100]

Why: Index 0 is the first item; assignment through index 2 replaces 30 with 100 (lists are mutable).

Q2.

```
colors = ['red', 'green', 'blue', 'yellow']
print(colors[-1])
print(colors[1:3])
```

Answer: yellow | ['green', 'blue']

Why: -1 is the last item; slice stops BEFORE index 3.

Q3.

```
total = 0
for num in [5, 10, 15]:
    total += num
print("Sum is:", total)
```

Answer: Sum is: 30

Why: Accumulator pattern: 0+5=5, 5+10=15, 15+15=30.

Q4.

```
numbers = [3, 8, 1, 9, 4]
max_val = numbers[0]
for num in numbers:
    if num > max_val:
        max_val = num
print("Max is:", max_val)
```

Answer: Max is: 9

Why: max_val: 3 → 8 → 8 → 9 → 9.

Q5.

```
count = 0
for num in [2, 7, 4, 9, 6]:
    if num % 2 == 0:
        count += 1
print("Even count:", count)
```

Answer: Even count: 3

Why: 2, 4 and 6 are even; 7 and 9 are odd.

Q6.

```
def find_average(numbers):
    total = sum(numbers)
    count = len(numbers)
    return total / count
```

```
avg = find_average([80, 90, 100])
print(avg)
```

Answer: 90.0

Why: $270 / 3 = 90.0$ — note the .0 because / always gives a float.

Q7.

```
nums = [1, 2, 3]
for i in range(len(nums)):
    print("Index:", i, "Value:", nums[i])
```

Answer: Index: 0 Value: 1 | Index: 1 Value: 2 | Index: 2 Value: 3

Why: range(len(nums)) yields 0, 1, 2.

Q8.

```
def is_over_budget(total, limit):
    return total > limit

print(is_over_budget(500, 400))
print(is_over_budget(300, 400))
```

Answer: True | False

Why: The comparison itself evaluates to a Boolean, which is returned directly.

Part F — Error-Identification & Debugging (6) | දෝෂ හඳුනාගැනීම සහ නිදොස් කිරීම

For each snippet: (a) name the error type (syntax / runtime / logical), (b) explain the cause, (c) write the corrected code.

(අ) දෝෂ වර්ගය, (ආ) හේතුව, (ඇ) නිවැරදි කේතය ලියන්න.

Q1.

```
nums = [1, 2, 3, 4, 5]
print(nums[5])
```

(a) Error type: Runtime error — IndexError.

(b) Cause: Length is 5, so valid indices are 0–4; index 5 does not exist.

(c) Corrected code:

```
print(nums[4]) # or nums[-1]
```

Q2.

```
for num in numbers
    print(num)
```

(a) Error type: Syntax error — missing colon.

(b) Cause: Compound statements (for/if/def) must end with ':'. Python reports this before running.

(c) Corrected code:

```
for num in numbers:
    print(num)
```

Q3.

```
marks = [40, 55, 70]
total = 0
for m in marks:
    total += m
avg = total / 2
print(avg)
```

(a) Error type: Logical error.

(b) Cause: The program runs, but dividing by 2 instead of len(marks) gives the wrong average (82.5 instead of 55.0). Logical errors are never reported by Python.

(c) Corrected code:

```
avg = total / len(marks)
```

Q4.

```
max_val = 0
for num in [-5, -2, -9]:
    if num > max_val:
        max_val = num
print(max_val) # prints 0
```

(a) Error type: Logical error.

(b) Cause: Starting max_val at 0 fails for all-negative lists: no element beats 0, so 0 is wrongly printed instead of -2.

(c) Corrected code:

```
max_val = numbers[0]    # assume first item is largest
```

Q5.

```
i = 0
while i < len(nums):
    print(nums[i])
```

(a) Error type: Logical/runtime error — infinite loop.

(b) Cause: *i* is never updated, so the condition *i* < len(nums) stays True forever.

(c) Corrected code:

```
i = 0
while i < len(nums):
    print(nums[i])
    i += 1
```

Q6.

```
scores = []
avg = sum(scores) / len(scores)
```

(a) Error type: Runtime error — ZeroDivisionError.

(b) Cause: len([]) is 0; dividing by zero crashes at run time. Always check for the empty list first (lecture: a common mistake before finding Max/averages).

(c) Corrected code:

```
if scores:
    avg = sum(scores) / len(scores)
else:
    print("List is empty")
```

Part G — Algorithm & Pseudocode Questions (4) | ඇල්ගොරිතම / ව්‍යාජ කේත ප්‍රශ්න

Q1. Write pseudocode to calculate the average of a list of marks.

ලකුණු ලැයිස්තුවක සාමාන්‍යය ගණනය කිරීමට ව්‍යාජ කේතය ලියන්න.

Model answer:

```
1. START
2. SET TOTAL to 0
3. FOR each MARK in LIST
4.     ADD MARK to TOTAL
5. PRINT TOTAL / LENGTH
6. END
```

Q2. Write pseudocode to find the largest number in a list.

විශාලතම අගය සෙවීමට ව්‍යාජ කේතය ලියන්න.

Model answer:

```
1. START
2. SET MAX to first item of LIST
3. FOR each NUM in LIST
4.     IF NUM > MAX THEN SET MAX to NUM
5. PRINT MAX
6. END
```

Q3. Write pseudocode to count how many marks in a list are 50 or above (pass count).

සමත් ගණන (50 හෝ වැඩි) ගණන් කිරීමට ව්‍යාජ කේතය.

Model answer:

```
1. START
2. SET COUNT to 0
3. FOR each MARK in LIST
4.     IF MARK >= 50 THEN ADD 1 to COUNT
5. PRINT COUNT
6. END
```

Q4. Write the algorithm (plain numbered steps) for the daily Git workflow of saving work and sharing it with the team.

දෛනික Git කාර්ය ප්‍රවාහය පියවර ලෙස ලියන්න.

Model answer:

```
1. START
2. EDIT files in the working directory (VS Code)
3. CHECK state with git status
4. STAGE changes: git add .
5. COMMIT snapshot: git commit -m "message"
6. SYNC first: git pull origin main
7. PUSH to remote: git push origin main
8. END – team sees the code on GitHub
```

Part H — Flowchart Questions (2) | ගැලීම් සටහන් ප්‍රශ්න

Standard symbols: oval = Start/End, parallelogram = Input/Output, rectangle = Process, diamond = Decision, arrows = flow.

සම්මත සංකේත: ඔවලය = ආරම්භය/අවසානය, සමාන්තරාස්‍රය = ආදාන/ප්‍රතිදාන, සෘජුකෝණාස්‍රය = ක්‍රියාවලිය, දියමන්තිය = තීරණය.

Q1. Draw a flowchart for the count-evens algorithm.

ඉරට්ටේ ගණන ඇල්ගොරිතමය සඳහා ගැලීම් සටහනක් අඳින්න.

Model answer (box-by-box):

```
(START) oval
|
[COUNT = 0] rectangle
|
<More items in LIST?> diamond — No —> /PRINT COUNT/ —> (END)
| Yes
[Take next NUM] rectangle
|
<NUM % 2 == 0 ?> diamond — No —┐
| Yes                             |
[COUNT = COUNT + 1]             |
└────────────────────────────────┘
                                back to <More items?> ◀
```

Marking points: initialisation box before the loop; loop-condition diamond; inner decision diamond for the modulo test; output only after the loop ends; arrows returning to the loop test.

Q2. Draw a flowchart for grading a single mark: 50 or above → “Pass”, otherwise “Fail”.

තනි ලකුණකට: 50 හෝ වැඩි නම් “Pass”, නැතිනම් “Fail”.

Model answer (box-by-box):

```
(START)
|
/INPUT MARK/           parallelogram
|
<MARK >= 50 ?>         diamond
Yes |                   | No
/PRINT "Pass"/         /PRINT "Fail"/
  \                   /
   (END)
```

Marking points: single input; one decision diamond with two labelled exits (Yes/No); both paths joining before END.

Part I — Short Programming Questions (5) | කෙටි ක්‍රමලේඛන ප්‍රශ්න

Q1. Create a list of 5 colors. Print the 2nd and 4th colors, then print the total number of colors. (Lecture Exercise: Basic Lists)

වර්ණ 5ක ලැයිස්තුවක් සාදන්න; 2 වන සහ 4 වන වර්ණ මුද්‍රණය කරන්න; මුළු ගණන මුද්‍රණය කරන්න.

Model solution:

```
colors = ["red", "green", "blue", "yellow", "white"]
print(colors[1])    # 2nd color -> green (index 1!)
print(colors[3])    # 4th color -> yellow
print(len(colors))  # 5
```

Q2. Write a program that finds the sum of all elements, the maximum, and the count of even numbers in nums = [12, 7, 20, 3, 8]. (Lecture Exercise: Processing Data)

එකතුව, උපරිමය සහ ඉරවිටේ ගණන සොයන වැඩසටහනක් ලියන්න.

Model solution:

```
nums = [12, 7, 20, 3, 8]

total = 0
max_val = nums[0]
count = 0
for num in nums:
    total += num
    if num > max_val:
        max_val = num
    if num % 2 == 0:
        count += 1

print("Sum is:", total)    # 50
print("Max is:", max_val)  # 20
print("Even count:", count) # 3
```

Q3. Print a list in reverse order, find the second-largest number, and remove all duplicates from nums = [4, 9, 1, 9, 6, 4]. (Lecture Exercise: Transformation)

ප්‍රතිලෝමව මුද්‍රණය; දෙවන විශාලතම අගය; අනුපිටපත් ඉවත් කිරීම.

Model solution:

```
nums = [4, 9, 1, 9, 6, 4]

print(nums[::-1])    # [4, 6, 9, 1, 9, 4] reverse

unique = list(set(nums))    # remove duplicates -> [1, 4, 6, 9]
unique.sort()
print(unique)

runner_up = sorted(set(nums))[-2]
print("Second largest:", runner_up)    # 6
```

Q4. Write a function find_average(numbers) that returns the average of a list, then call it with [80, 90, 100] and print the result.

සාමාන්‍යය ලබා දෙන ශ්‍රිතයක් ලියා [80, 90, 100] සමඟ අමතන්න.

Model solution:

```
def find_average(numbers):
    total = sum(numbers)
```

```
count = len(numbers)
return total / count

avg = find_average([80, 90, 100])
print(avg)    # 90.0
```

Q5. Homework: Expense Analyzer. Store daily expenses in a list; calculate total spending; find the single highest expense; create a function `is_over_budget(total, limit)` that returns True/False.

ගෙවීම් විශ්ලේෂකය: එකතුව, ඉහළම වියදම, `is_over_budget` ශ්‍රිතය.

Model solution:

```
expenses = [1200, 450, 3200, 800, 1500]

total = 0
for e in expenses:
    total += e
print("Total spending:", total)    # 7150

highest = expenses[0]
for e in expenses:
    if e > highest:
        highest = e
print("Highest expense:", highest)    # 3200

def is_over_budget(total, limit):
    return total > limit

print(is_over_budget(total, 5000))    # True
```


Part J — Substantial Practical Problem | විශාල ප්‍රායෝගික ගැටලුව

Student Performance System (25 marks)

Based on the Week 3 Group Activity. Analyze a list of 10 student marks and output: (a) the pass count (mark ≥ 50), (b) the average mark, and (c) the top mark. Requirements: (1) write the pseudocode / explain your logic [5 marks]; (2) implement a reusable list-processing function `analyze_marks(marks)` that RETURNS all three statistics [12 marks]; (3) handle the empty-list case safely [3 marks]; (4) call the function with a sample list of 10 marks and display the results with clear labels [5 marks].

ශිෂ්‍ය කාර්යසාධන පද්ධතිය: ලකුණු 10ක ලැයිස්තුවක් විශ්ලේෂණය කර සමත් ගණන (≥ 50), සාමාන්‍යය සහ ඉහළම ලකුණ ලබා දෙන්න. ව්‍යාජ කේතය, නැවත භාවිතා කළ හැකි ශ්‍රිතය, හිස් ලැයිස්තු ආරක්ෂාව සහ ලේඛ්‍ය සහිත ප්‍රතිදානය අවශ්‍යයි.

Step 1 — Logic / Pseudocode (5 marks) | තර්කය / ව්‍යාජ කේතය

```
1. START
2. IF LIST is empty THEN PRINT "No marks" and END
3. SET PASS_COUNT to 0, TOTAL to 0, TOP to first item
4. FOR each MARK in LIST
5.     ADD MARK to TOTAL
6.     IF MARK  $\geq 50$  THEN ADD 1 to PASS_COUNT
7.     IF MARK > TOP THEN SET TOP to MARK
8. SET AVERAGE to TOTAL / LENGTH of LIST
9. RETURN PASS_COUNT, AVERAGE, TOP
10. END
```

Step 2 — Full Model Solution (Python) | සම්පූර්ණ ආදර්ශ විසඳුම

```
def analyze_marks(marks):
    """Analyze a list of student marks.
    Returns (pass_count, average, top_mark)."""

    # (3) Safety: never index or divide an empty list
    if len(marks) == 0:
        return 0, 0, None

    pass_count = 0          # counter
    total = 0               # accumulator
    top = marks[0]          # assume first is largest

    for mark in marks:      # single O(n) traversal
        total += mark
        if mark  $\geq 50$ :
            pass_count += 1
        if mark > top:
            top = mark

    average = total / len(marks)
    return pass_count, average, top

# (4) Call with a sample list of 10 marks
class_marks = [45, 78, 62, 33, 90, 55, 47, 81, 69, 50]

passes, avg, best = analyze_marks(class_marks)

print("Pass count:", passes)      # 7
print("Average mark:", avg)       # 61.0
print("Top mark:", best)          # 90
```

Expected Output | අපේක්ෂිත ප්‍රතිදානය

Pass count: 7
Average mark: 61.0
Top mark: 90

Marking Scheme Notes | ලකුණු දීමේ සටහන්

Component	Marks	What examiners look for
Pseudocode / logic	5	Accumulator + counter initialised before loop; decisions inside the loop; average computed after the loop.
Function definition & return	12	def with a parameter; single traversal; correct pass (≥ 50), average (total/len), and top (start from marks[0]); values RETURNED, not just printed.
Empty-list handling	3	Guard before marks[0] and before division — avoids IndexError and ZeroDivisionError.
Function call & labelled output	5	List of 10 marks; tuple unpacking or indexing of the result; clear labels in print statements.

Extension idea (mentioned in the lecture's Student Marks System use case)

Input: list of marks → Processing: calculate average & max → Output: grade statistics. You can extend the function to also return the minimum mark or a fail count — the same single-loop pattern applies.

ආදානය: ලකුණු ලැයිස්තුව → සැකසුම: සාමාන්‍යය සහ උපරිමය → ප්‍රතිදානය: ශ්‍රේණි සංඛ්‍යාලේඛන.
අවම ලකුණ හෝ අසමත් ගණන ද එකම ලූප රටාවෙන් එකතු කළ හැක.